# LAPORAN PRAKTIKUM
# INTERNET OF THINGS


**Disusun oleh :**

**Nama : Restu anggara**

**NPM : 25782051**

**Kelas : TRI-3B**


**Dosen :**

**Dr.Ir.Septafiansyah Dwi Putra,S.T.,M.T.**

**Agus Ambarwari, S.Pd.,M.Kom.**

**Andi Saptono,S.Kom**

**TEKNOLOGI REKAYASA INTERNET**

**JURUSAN TEKNOLOGI INFORMASI**

**POLITEKNIK NEGERI LAMPUNG**

**2026/2027**

## 7.5.1 Praktikum 1: Pembangunan Ekosistem IoT Server dengan Docker Compose

- Struktur direktori

```
C:\Users\acer\iot-server>tree
Folder PATH listing
Volume serial number is EAFA-E5AC
C:.
├───mosquitto
│   ├───config
│   ├───data
│   └───log
└───notered
```

- Masuk ke direktori iot-server

```
C:\Users\acer>cd C:\users\acer\iot-server
```

- Menunggu kedua kontainer sampai selesai

```
C:\Users\acer\iot-server>docker compose up -d
[+] up 14/24
 - Image nodered/node-red:latest [▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒] Pulling
 - Image eclipse-mosquitto:latest [▒▒▒]                Pulling
```

- Proses selesai

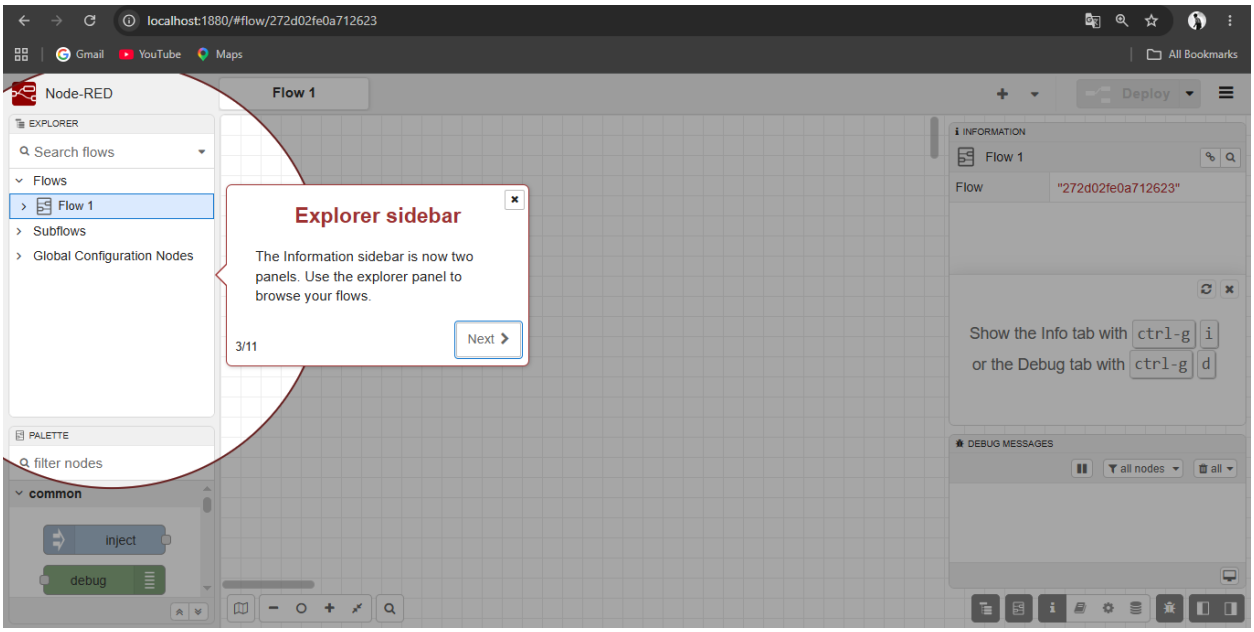
```
C:\Users\acer\iot-server>docker compose up -d
[+] up 24/26
[+] up 29/29ered/node-red:latest [▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒] 133.6MB / 235.4MB Pulling
 ▒ Image nodered/node-red:latest  Pulled                                    1649.8s
 ▒ Image eclipse-mosquitto:latest Pulled                                     474.4s
 ▒ Network iot-server_default     Created                                      0.3s
 ▒ Container mqtt-broker          Started                                     7.9s
 ▒ Container node-red             Started                                     8.6s
```

- Verifikasi kedua kontainer telah aktif dan berjalan normal

```
C:\Users\acer\iot-server>docker ps
CONTAINER ID   IMAGE                  COMMAND                  CREATED         STATUS                   PORTS
                     NAMES
cf1977863920   nodered/node-red:latest    "./entrypoint.sh"        3 minutes ago   Up 3 minutes (healthy)   0.0.0.0:1880->1880/tcp, [::]:1880->1880/tcp
                     node-red
387869d567df   eclipse-mosquitto:latest   "/docker-entrypoint.…"   3 minutes ago   Up 2 seconds             0.0.0.0:1883->1883/tcp, [::]:1883->1883/tcp, 0.0.0.0:9001->9
001/tcp, [::]:9001->9001/tcp   mqtt-broker
```

STATUS untuk kontainer mqtt-broker dan node-red menunjukkan Up (healthy/running).

- server IoT lokal telah siap beroperasi secara sempurna!




Praktikum:

Praktikum 1 berhasil membangun server IoT lokal terpadu menggunakan Docker Compose. Berkas docker-compose.yml mendefinisikan dua layanan, yaitu broker Eclipse Mosquitto (port 1883 untuk MQTT dan 9001 untuk WebSockets) dan Node-RED (port 1880), yang dijalankan sekaligus dengan satu perintah docker compose up -d. Konfigurasi broker diatur melalui mosquitto.conf, dan pemetaan volume menjaga data serta konfigurasi tetap tersimpan di laptop meskipun kontainer dihapus. Keberhasilan ditandai dengan kedua kontainer berstatus Up pada docker ps dan tampilnya antarmuka Node-RED di http://localhost:1880. Selama pengerjaan, kesalahan penulisan pada mosquitto.conf (misalnya nilai log_dest yang salah) terbukti dapat membuat kontainer broker terus restart, sehingga pembacaan docker logs menjadi langkah penting dalam mendiagnosis masalah.

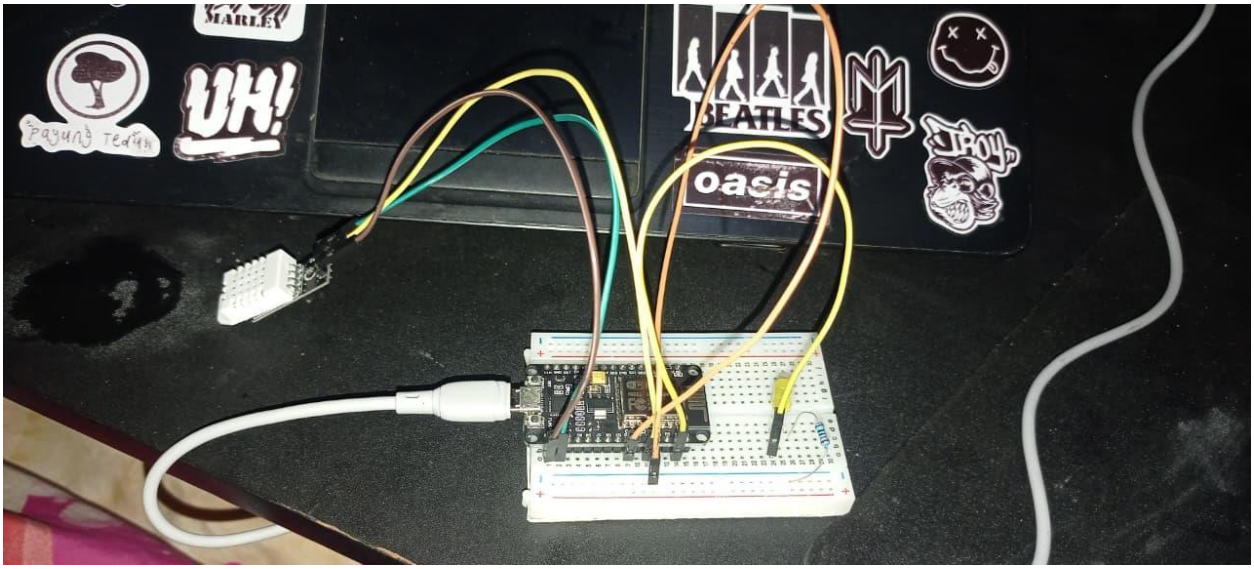
## 7.5.2 Praktikum 2: Migrasi Simpul NodeMCU ESP8266 ke Broker MQTT Lokal

Sensor DHT11/DHT22:

- Pin VCC → Pin 3V3 NodeMCU
- Pin GND → Pin GND NodeMCU
- Pin Data → Pin D4 (GPIO 2)

LED Indikator Aktuator:

- Anoda (kaki panjang) → Pin D1 (GPIO 5)
- Katoda (kaki pendek) → Resistor 220 Ω → Pin GND




- Ip dari ipconfig

```
Wireless LAN adapter Wi-Fi 2:

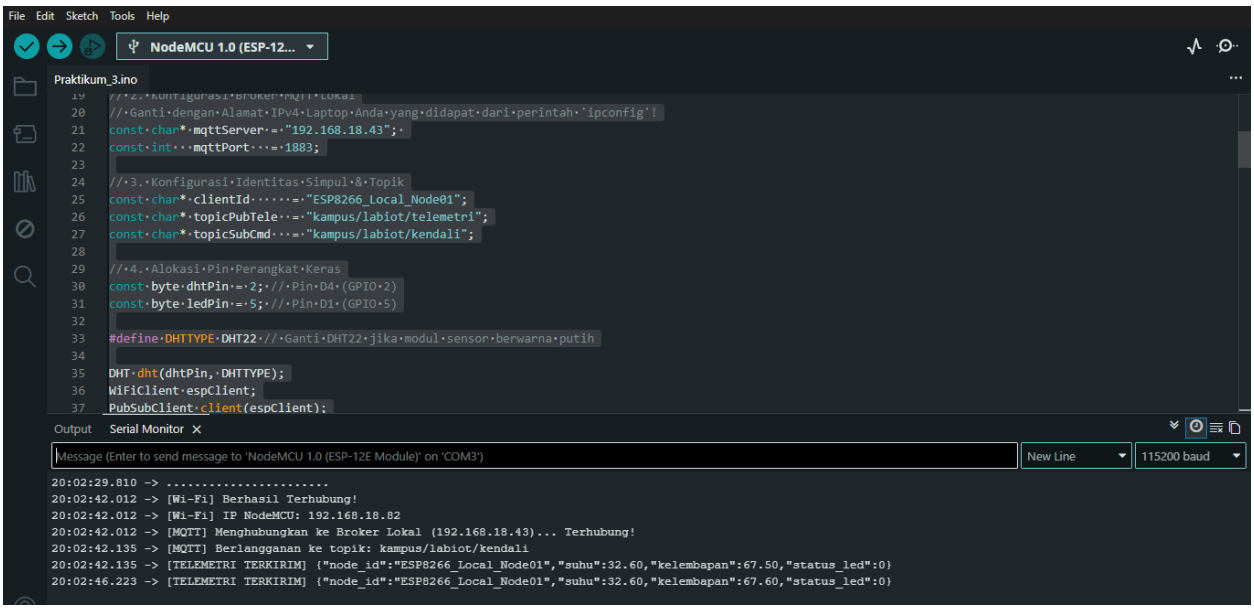
   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::cab6:fa46:34ed:528%28
   IPv4 Address. . . . . . . . . . . : 192.168.18.43
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : 192.168.18.1

Ethernet adapter vEthernet (Default Switch):

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::363e:2a63:5d03:7f29%52
   IPv4 Address. . . . . . . . . . . : 172.29.240.1
   Subnet Mask . . . . . . . . . . . : 255.255.240.0
   Default Gateway . . . . . . . . . :

C:\Users\acer>
```

- Hasil

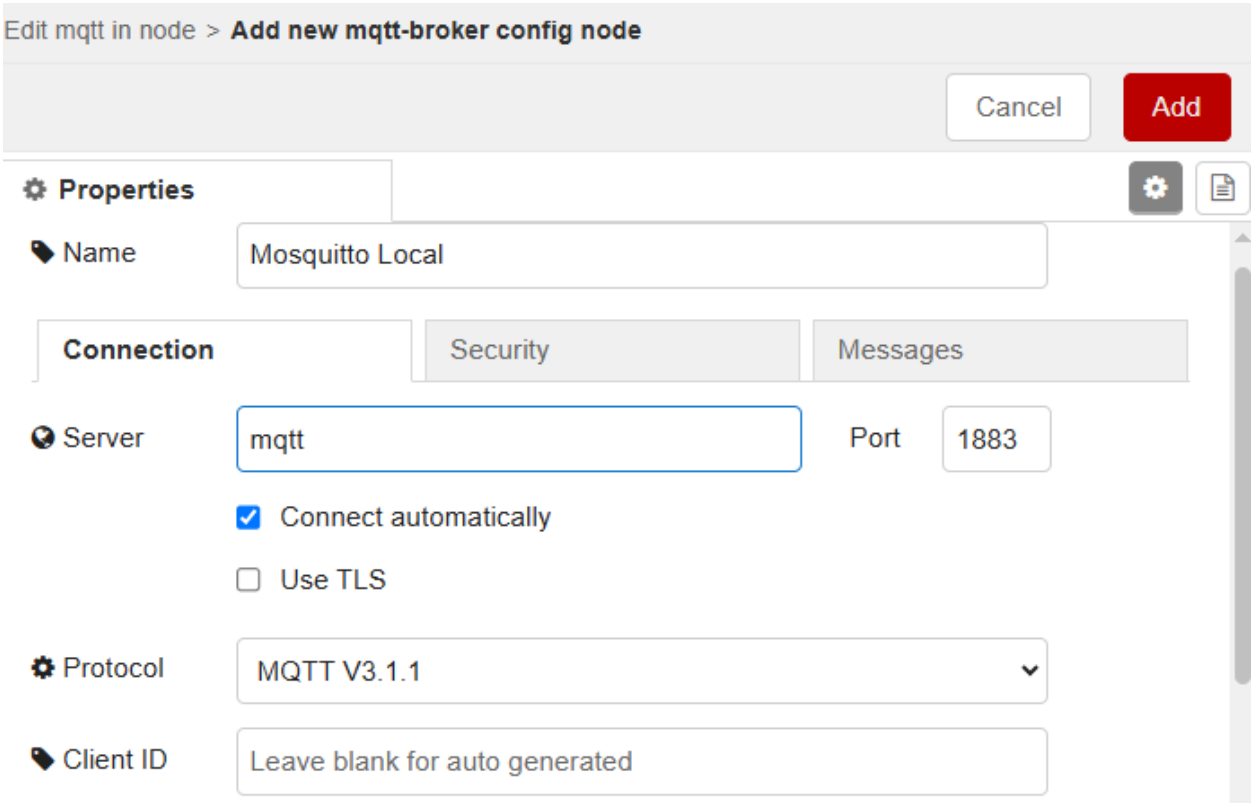



Kesimpulan:

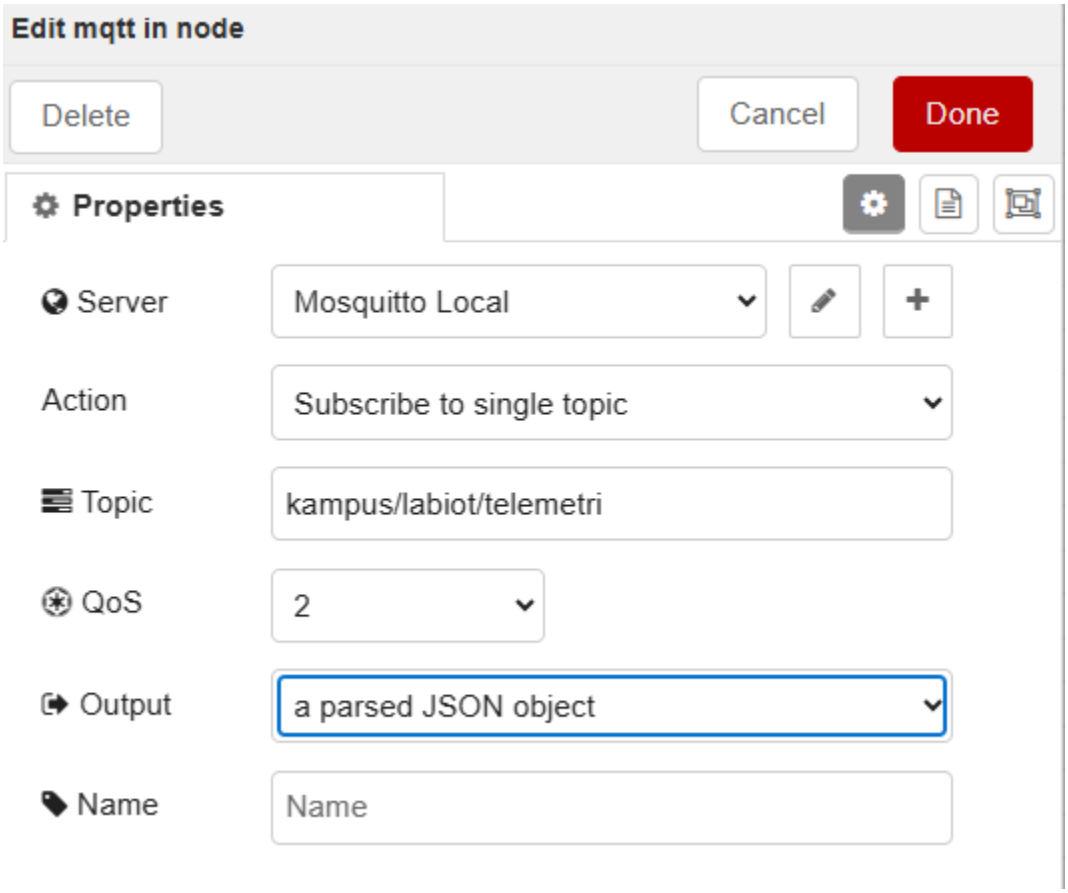
Praktikum 2 berhasil memigrasikan NodeMCU ESP8266 dari broker publik ke broker Mosquitto lokal di laptop. Sketsa diarahkan ke alamat IPv4 laptop pada port 1883, lalu mikrokontroler membaca sensor DHT di pin D4, mengirim telemetri JSON secara periodik setiap 4 detik, dan berlangganan ke topik kendali untuk menggerakkan LED di pin D1. Keberhasilan koneksi sangat bergantung pada kondisi jaringan: NodeMCU dan laptop harus berada dalam satu subnet, alamat yang dipakai harus IP laptop (bukan gateway), broker harus aktif, dan firewall tidak boleh memblokir port 1883. Kode status rc pada Serial Monitor, seperti rc=-2 yang menandakan broker tidak terjangkau, membantu menelusuri sumber kegagalan koneksi.

### 7.5.3 Praktikum 3: Integrasi Alur Data Telemetri Dua Arah pada Node-RED

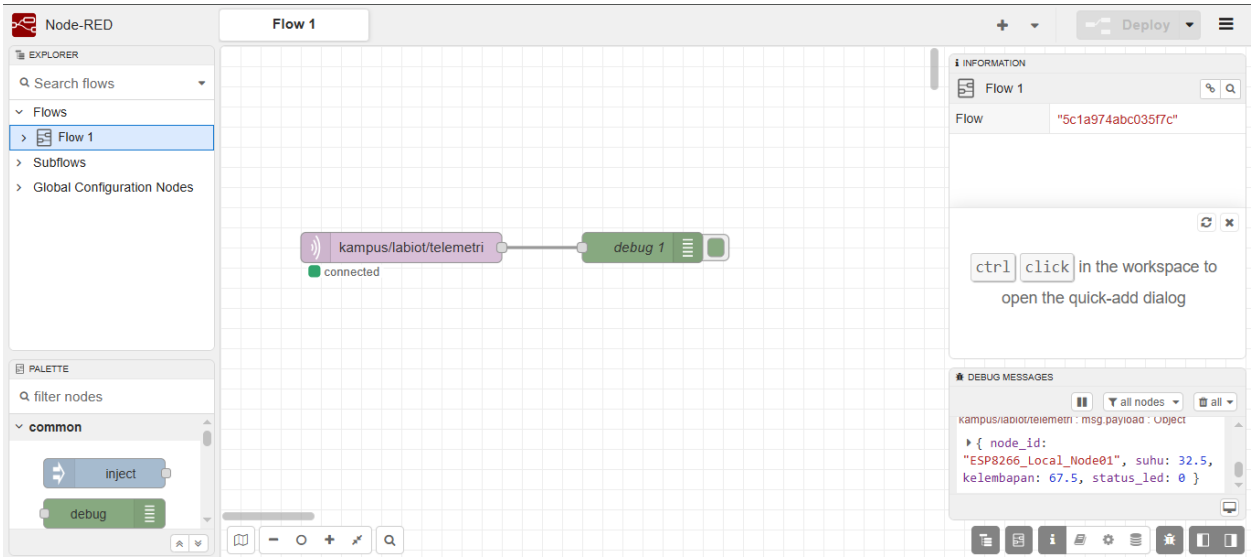
Menambahkan Konfigurasi Server MQTT Broker



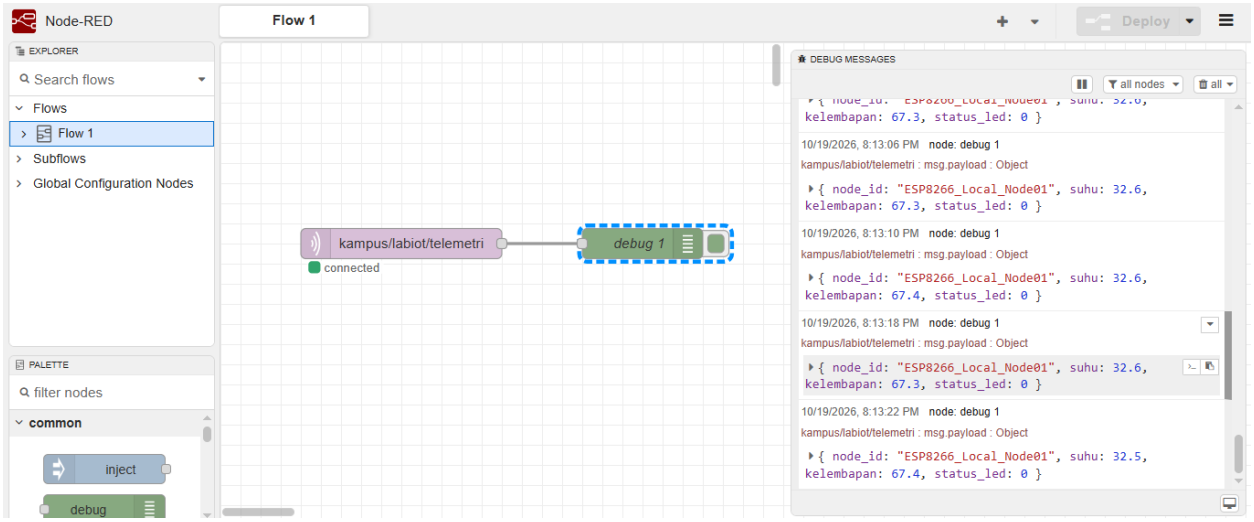
Mengonfigurasi Parameter Topik




Menerapkan Alur Kerja (*Deploy*)



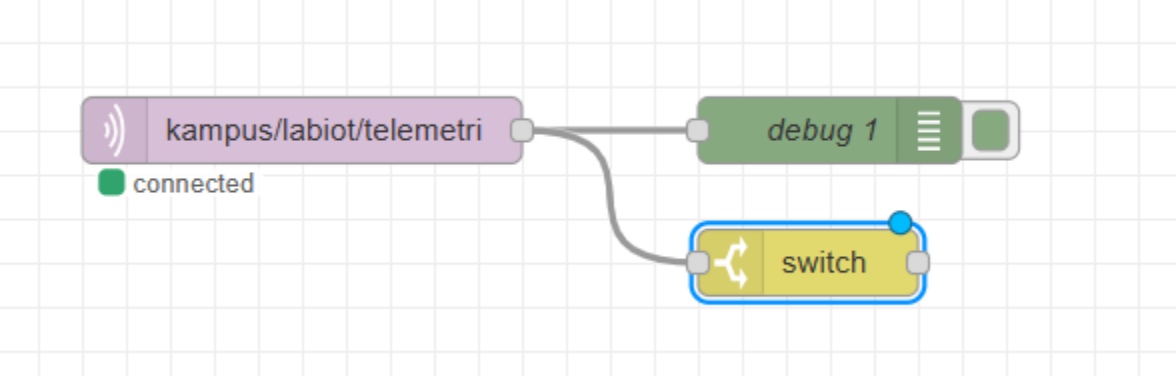
Menganalisis Aliran Data



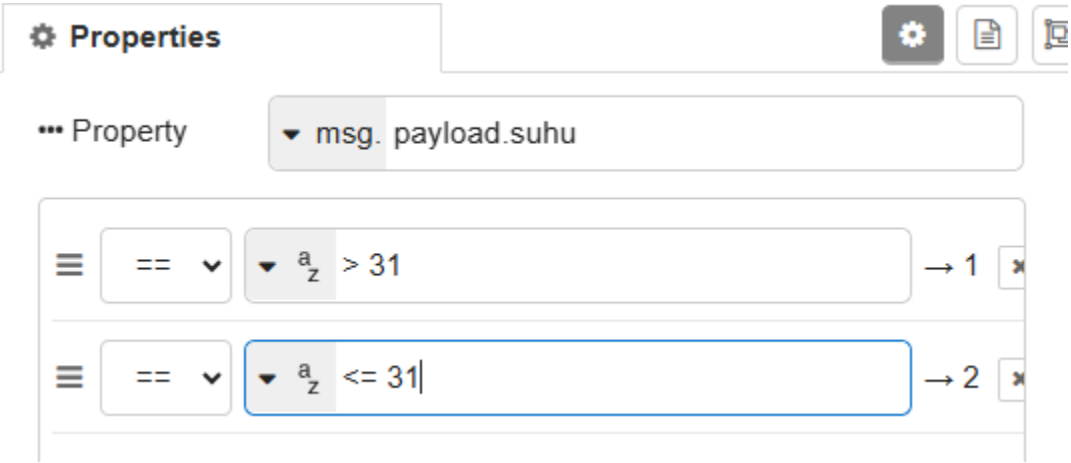

Kesimpulan:

Praktikum 3 berhasil menunjukkan bahwa Node-RED dapat menangkap dan membedah aliran telemetri dari NodeMCU melalui broker Mosquitto lokal. Node mqtt in berlangganan ke topik kampus/labiot/telemetri dengan output berupa *parsed JSON object*, sehingga paket JSON langsung terbaca sebagai objek berisi node_id, suhu, kelembapan, dan status_led yang tampil di panel Debug setiap 4 detik. Alamat server broker diisi dengan nama layanan mqtt, bukan localhost atau IP laptop, karena Node-RED dan Mosquitto berada dalam satu jaringan Docker Compose dan saling menemukan lewat nama layanan. Alur dasar ini menjadi fondasi untuk membangun logika otomasi pada praktikum berikutnya.

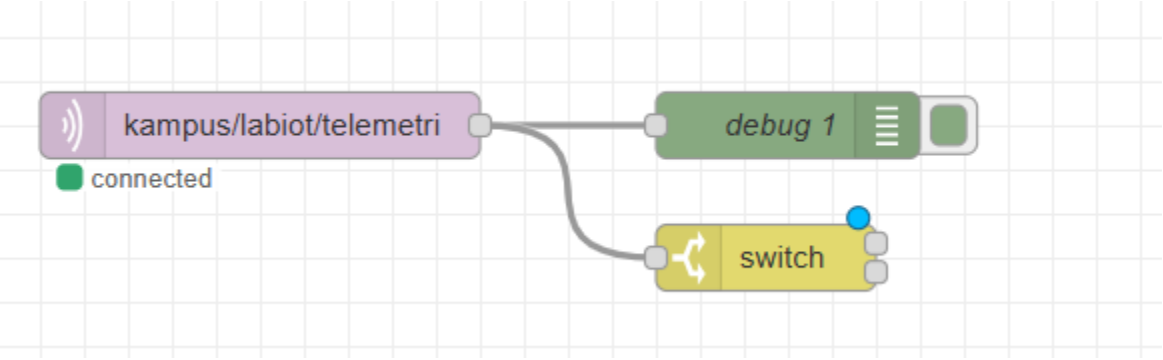
### 7.5.4 Praktikum 4: Implementasi Rule Engine Otomasi Aktuator Loop Tertutup



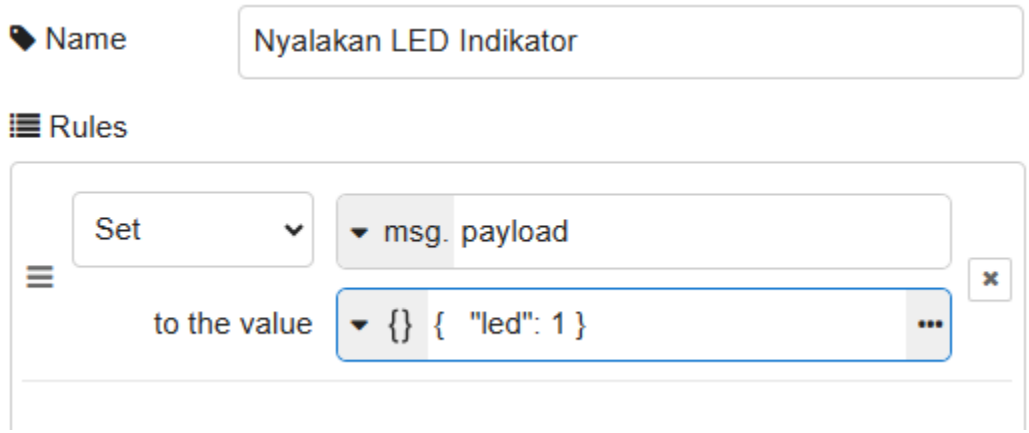

- Mengubah msg jadi payload.suhu dan membuat 2 aturan.



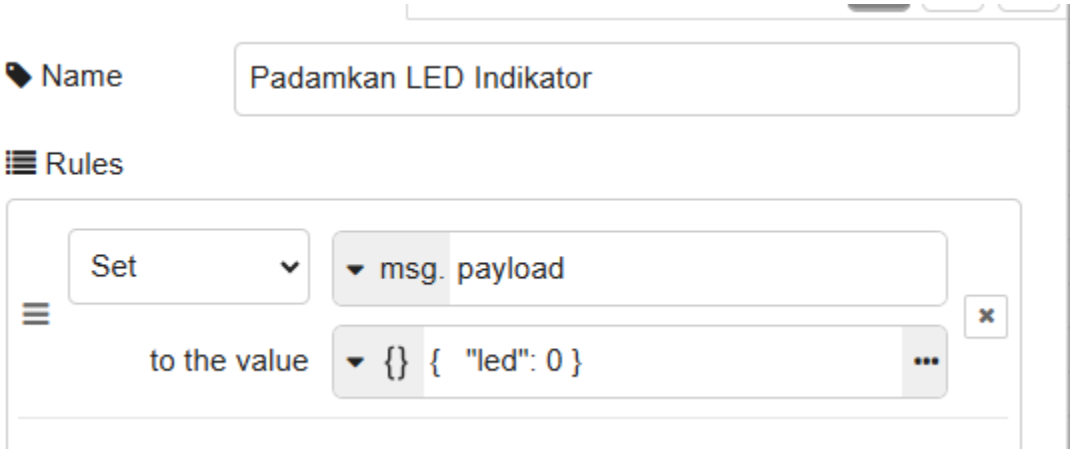

- Kini switch memliki 2 port



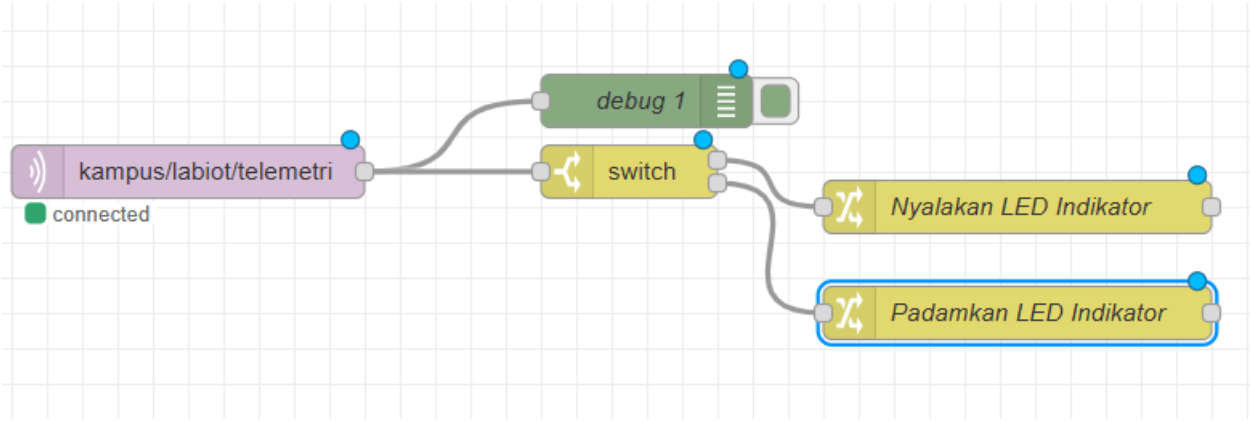
- Aturan untuk menyalakan led



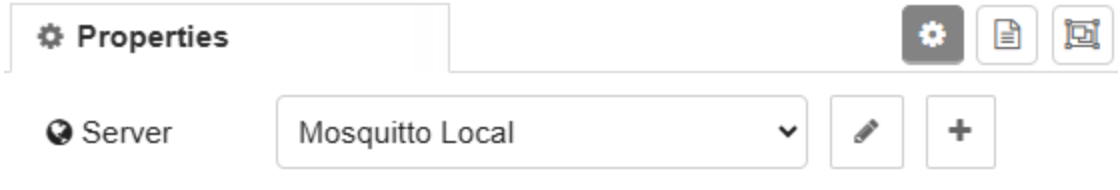

- Aturan untuk menyalakan led

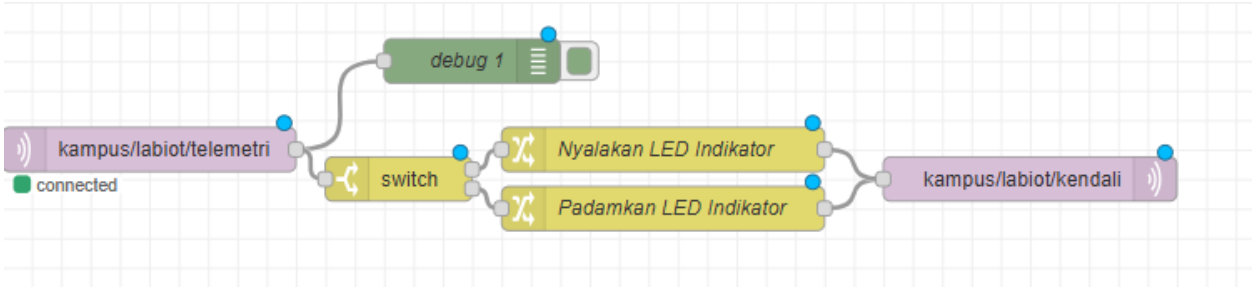



- Hasil



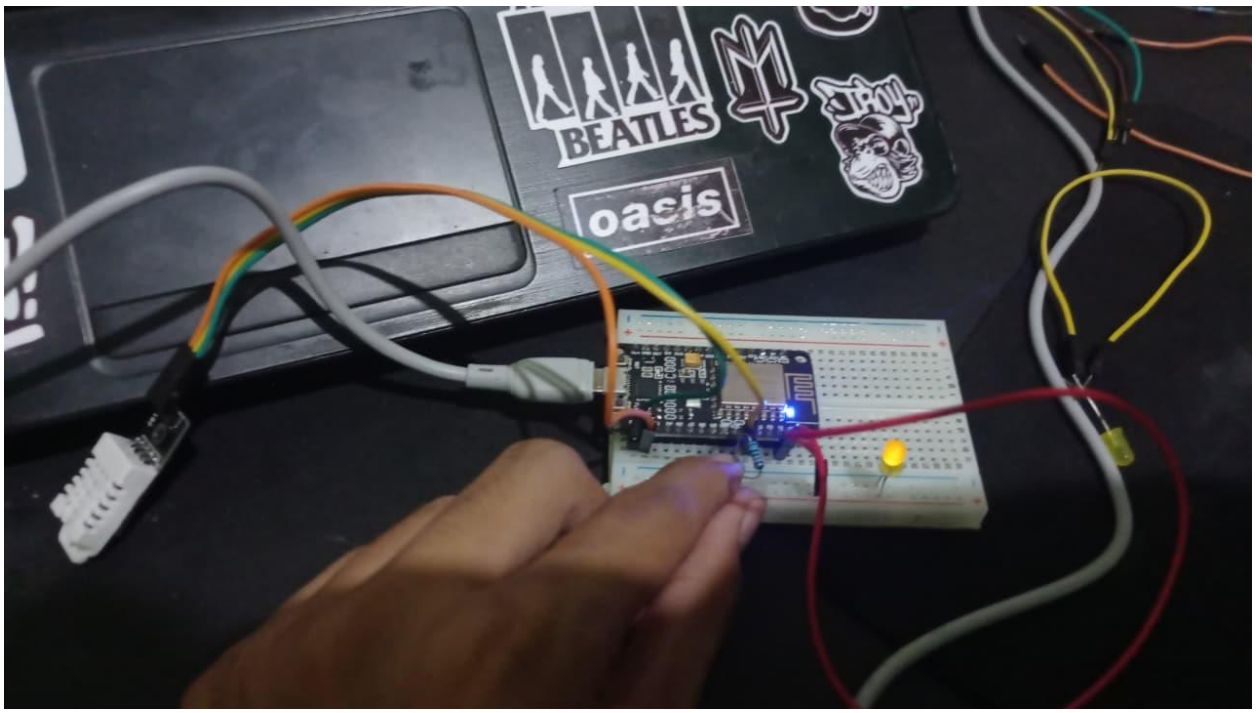
- Menerbitkan Instruksi Balik ke Broker (mqtt out)




Successfully deployed

- Pengujian Fisik Sistem Otomasi




Kesimpulan:

Rule engine loop tertutup berhasil dibangun di Node-RED: telemetri suhu dari NodeMCU dievaluasi dengan node switch pada ambang 31 °C, lalu node change dan mqtt out mengirim perintah {"led": 1} atau {"led": 0} sehingga LED menyala otomatis saat suhu melewati 31 °C dan padam saat turun kembali. Logika keputusan berada di server sehingga NodeMCU tidak perlu diprogram ulang, meski kendali dua posisi ini masih rentan membuat LED berkedip saat suhu berfluktuasi di sekitar ambang batas.

### 7.6 Latihan

- **Latihan 1 (Analisis Jaringan Internal Docker & Service Discovery):**
  Pada Praktikum 3, di dalam konfigurasi node mqtt in pada Node-RED, kita mengisi alamat server broker dengan nama teks **mqtt**, bukan alamat IP lokal laptop (192.168.1.15) maupun localhost.

  i. Mengapa penulisan localhost pada Node-RED di dalam kontainer Docker akan menyebabkan kegagalan koneksi (*Connection Refused*)?

  localhost mengacu pada kontainer itu sendiri, bukan laptop atau kontainer lain. Jadi, ketika Node-RED menggunakan localhost sebagai alamat broker MQTT, Node-RED akan mencari layanan MQTT di dalam kontainernya sendiri. Karena broker MQTT berada di kontainer yang berbeda, koneksi akan gagal dan muncul pesan Connection Refused.

  ii. Jelaskan bagaimana mekanisme *Docker Embedded DNS Server* dan *Bridge Network* bekerja sehingga kontainer node-red dapat menemukan kontainer mqtt-broker cukup dengan memanggil nama layanannya!

  - Docker Embedded DNS Server: Docker menyediakan DNS internal yang menerjemahkan nama layanan seperti mqtt menjadi alamat IP internal kontainer MQTT Broker. Dengan demikian, Node-RED tidak perlu mengetahui atau menuliskan alamat IP secara manual.

  - Bridge Network: Bridge Network memungkinkan kontainer yang tergabung dalam jaringan Docker yang sama untuk berkomunikasi satu sama lain secara langsung menggunakan jaringan internal Docker.

  Alur koneksinya:

  1. Node-RED meminta koneksi ke alamat mqtt:1883.
  2. Docker Embedded DNS Server menerjemahkan nama mqtt menjadi alamat IP internal kontainer MQTT Broker.
  3. Bridge Network meneruskan komunikasi dari kontainer Node-RED ke kontainer MQTT Broker.
  4. MQTT Broker menerima koneksi pada port 1883 dan komunikasi dapat berlangsung.

- **Latihan 2 (Penanganan Masalah Konektivitas Jaringan & Subnet Firewall):**
  Seorang mahasiswa mencoba menghubungkan NodeMCU ESP8266 ke broker lokal di laptopnya. ESP8266 berhasil terhubung ke Wi-Fi dan mendapatkan alamat IP 192.168.43.102, sedangkan laptop memiliki alamat IP 192.168.1.15.

  i. Mengapa NodeMCU tidak akan pernah berhasil memublikasikan data ke broker Mosquitto pada kondisi tersebut?

NodeMCU ESP8266 memiliki alamat IP 192.168.43.102, sedangkan laptop memiliki alamat IP 192.168.1.15. Kedua perangkat berada pada subnet yang berbeda jika menggunakan subnet mask umum 255.255.255.0 (/24).

Akibatnya, NodeMCU tidak dapat langsung berkomunikasi dengan laptop melalui jaringan lokal tanpa adanya perangkat atau konfigurasi routing yang menghubungkan kedua subnet tersebut. Oleh karena itu, NodeMCU akan gagal mengirim data ke broker Mosquitto pada laptop.

ii. Langkah teknis apa yang harus dilakukan mahasiswa tersebut agar kedua perangkat berada dalam domain *broadcast / subnet* yang sama?

langkah-langkah berikut:

1. Hubungkan kedua perangkat ke jaringan Wi-Fi yang sama, misalnya melalui hotspot laptop.
2. Pastikan alamat IP keduanya berada pada subnet yang sama. Contohnya:
   - Laptop: 192.168.43.15
   - NodeMCU ESP8266: 192.168.43.102
   - Subnet mask: 255.255.255.0
3. Pastikan broker Mosquitto berjalan pada laptop dan menerima koneksi dari jaringan, bukan hanya dari localhost.
4. Izinkan koneksi masuk pada port MQTT 1883 melalui firewall laptop.
5. Atur alamat broker pada program ESP8266 menggunakan IP laptop, misalnya 192.168.43.15, bukan localhost.

- **Latihan 3 (Implementasi Filter Histeresis pada Rule Engine):**
  Pada sistem kendali dua posisi (*Bang-Bang*), jika suhu berfluktuasi cepat di sekitar ambang batas 31.0∘C (misal: 31.05∘C→30.95∘C→31.02∘C), lampu LED indikator akan berkedip-kedip cepat (*flickering / hunting*) yang mengganggu stabilitas visual sistem. Rancang modifikasi alur Node-RED menggunakan **function node** berbahasa JavaScript yang menerapkan logika **Histeresis**:
  - LED baru menyala jika suhu >31.5∘C (msg.payload = {"led": 1}).
  - LED baru padam jika suhu mendingin hingga <30.0∘C (msg.payload = {"led": 0}).
    Tuliskan kode JavaScript yang Anda masukkan ke dalam *Function Node* tersebut!

    1. Tambahkan variabel global

```
// Batas histeresis
const float BATAS_ATAS  = 31.5;  // LED menyala jika suhu > 31.5
const float BATAS_BAWAH = 30.0;  // LED padam jika suhu < 30.0
bool ledState = false;           // Menyimpan status LED terakhir
```

2. Mengubah pada bagian pembacaan sensor di loop()

```
if (!isnan(t) && !isnan(h)) {

  // ---- Logika Histeresis ----
  if (t > BATAS_ATAS) {
    ledState = true;
  } else if (t < BATAS_BAWAH) {
    ledState = false;
  }
  // Jika 30.0 <= t <= 31.5: status LED tidak berubah
  digitalWrite(ledPin, ledState ? HIGH : LOW);

  StaticJsonDocument<256> docOut;
  docOut["node_id"]    = clientId;
```

- **Latihan 4 (Analisis Mekanisme Volume Mount & Persistensi Data):**
  Pada berkas docker-compose.yml, terdapat pemetaan direktori - ./nodered:/data dan - ./mosquitto/data:/mosquitto/data.

  i. Apa yang akan terjadi pada alur kanvas visual Node-RED yang telah Anda buat jika kontainer dimatikan dengan perintah docker compose down, kemudian dinyalakan kembali, seandainya baris volume tersebut dihapus dari docker-compose.yml?

  Jika baris volume ./nodered:/data dihapus, maka alur (flow) yang telah dibuat pada Node-RED berisiko hilang ketika menjalankan perintah docker compose down, lalu docker compose up kembali.

  Hal ini terjadi karena docker compose down menghapus kontainer. Tanpa penyimpanan persisten, data Node-RED hanya tersimpan di dalam kontainer tersebut. Ketika kontainer baru dibuat, Node-RED akan kembali ke kondisi awal.

  Begitu juga dengan Mosquitto, tanpa volume ./mosquitto/data:/mosquitto/data, data broker yang tersimpan di dalam kontainer dapat hilang saat kontainer dibuat ulang.

  ii. Jelaskan perbedaan mendasar antara *Bind Mount* dan *Named Volume* pada Docker!

| Aspek | Bind Mount | Named Volume |
|---|---|---|
| Pengertian | Menghubungkan folder komputer host ke dalam kontainer. | Penyimpanan yang dibuat dan dikelola oleh Docker. |
| Lokasi | Ditentukan pengguna. | Dikelola otomatis oleh Docker. |
| Pengelolaan | Dikelola langsung melalui folder host. | Dikelola menggunakan perintah Docker. |
| Contoh | `./nodered:/data` | `nodered_data:/data` |
| Kegunaan | Memudahkan akses dan pengeditan file dari host. | Cocok untuk penyimpanan data aplikasi secara persisten. |

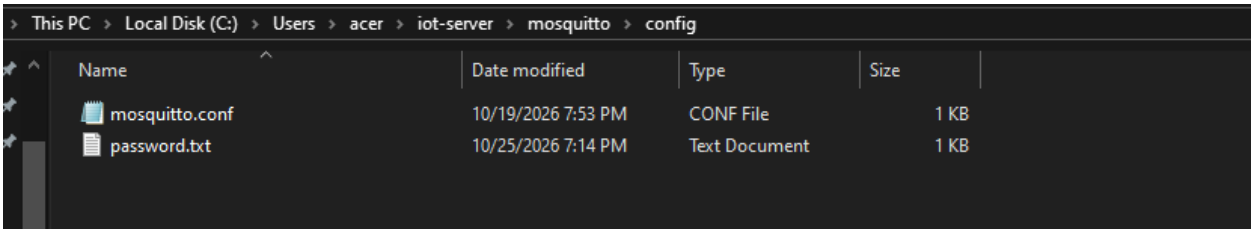
## 7.7 Tugas: Pengamanan Server IoT Industri (MQTT Authentication & Access Control)

1. Membuat Berkas Kredensial Terenkripsi di Dalam Kontainer

```
PS C:\Users\acer> docker exec -it mqtt-broker mosquitto_passwd -c /mosquitto/config/password.txt admin
Password:
Reenter password:
Adding password for user admin

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image →docker debug mqtt-broker
    Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\acer>
```

File password di folder config




2. Memperbarui Konfigurasi Kebijakan Mosquitto

```
# Perizinan Akses Awal (Tanpa Password)
allow_anonymous false
password_file /mosquitto/config/password.txt
```

3. Memuat Ulang Layanan Broker

```
PS C:\Users\acer> docker restart mqtt-broker
mqtt-broker
PS C:\Users\acer>
```

4. Analisis Dampak Kegagalan Sistem (Observasi Masalah)

```
19:28:24.130 -> [MQTT] Menghubungkan ke Broker Lokal (10.27.198.1)... Gagal (rc=-2). Coba lagi dalam 4 detik...
19:28:33.133 -> [MQTT] Menghubungkan ke Broker Lokal (10.27.198.1)... Gagal (rc=-2). Coba lagi dalam 4 detik...
19:28:42.207 -> [MQTT] Menghubungkan ke Broker Lokal (10.27.198.1)... Gagal (rc=-2). Coba lagi dalam 4 detik...
19:28:51.178 -> [MQTT] Menghubungkan ke Broker Lokal (10.27.198.1)... Gagal (rc=5). Coba lagi dalam 4 detik...
19:28:58.019 -> [MQTT] Menghubungkan ke Broker Lokal (10.27.198.1)... Gagal (rc=5). Coba lagi dalam 4 detik...
19:29:01.996 -> [MQTT] Menghubungkan ke Broker Lokal (10.27.198.1)... Gagal (rc=5). Coba lagi dalam 4 detik...
19:29:15.792 -> [MQTT] Menghubungkan ke Broker Lokal (10.27.198.1)... Gagal (rc=5). Coba lagi dalam 4 detik...
19:29:21.856 -> [MQTT] Menghubungkan ke Broker Lokal (10.27.198.1)... Gagal (rc=5). Coba lagi dalam 4 detik...
```

Kode status rc=5 merupakan representasi standar MQTT untuk 'Connection Refused: Not Authorized

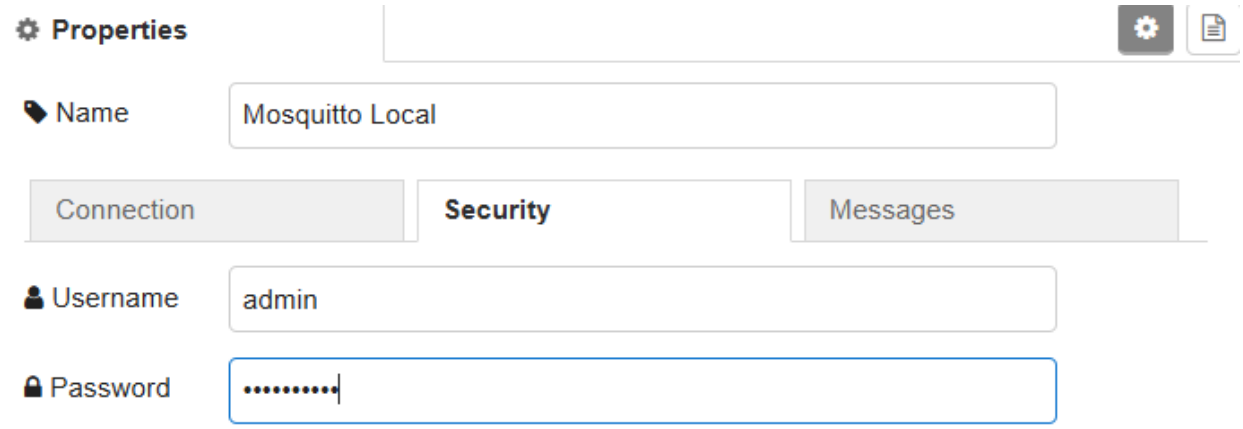
5. Rekayasa Perbaikan Sistem

Pemutakhiran Skrip Program NodeMCU ESP8266

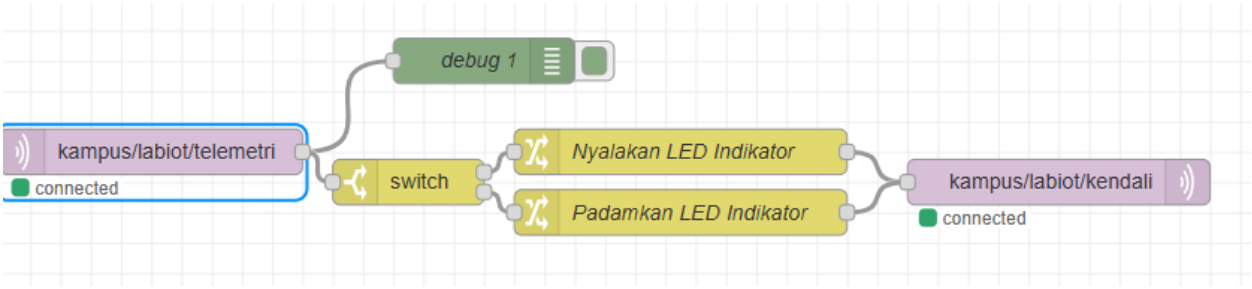
```
    // Format: client.connect(clientId, username, password)
    if (client.connect(clientId, "admin", "rahasia123")) {
      Serial.println(" Terhubung dengan Autentikasi Sah!");
      client.subscribe(topicSubCmd);
      Serial.printf("[MQTT] Berlangganan ke topik: %s\n", topicSubCmd);
    } else {
      Serial.printf(" Gagal (rc=%d). Coba lagi dalam 4 detik...\n", client.state());
      delay(4000);
    }
  }
}
```

Pemutakhiran Konfigurasi Kredensial pada Node-RED

nama broker Mosquitto Local dan username admin dan password rahasia123



Hasil nya




ndikator node MQTT pada kanvas Node-RED seketika kembali berwarna hijau (**connected**), dan aliran data telemetri serta kendali otomasi indikator LED loop tertutup kembali berfungsi normal dan aman dari gangguan pihak luar.

Kesimpulan:

Berdasarkan hasil Tugas 7.7, server IoT lokal berhasil diamankan dengan autentikasi username dan password. Berkas password.txt dibuat dengan mosquitto_passwd, lalu mosquitto.conf diatur dengan allow_anonymous false dan password_file, sehingga broker Mosquitto hanya menerima klien dengan kredensial yang sah dan menolak akses anonim dengan kode rc=5. Setelah kredensial admin ditambahkan pada client.connect() di ESP8266 dan pada tab Security di konfigurasi broker Node-RED, koneksi pulih dan alur telemetri serta kendali LED otomatis dapat berjalan kembali dengan aman. Meski begitu, autentikasi ini masih tergolong keamanan dasar karena kredensial dikirim tanpa enkripsi, sehingga untuk penerapan industri perlu ditambahkan TLS/SSL dan pembatasan hak akses per topik (ACL).